



AgentX — National Level Hackathon 2026

PROBLEM STATEMENT

Smart Campus Multi-Agent AI System

Background

Educational institutions today rely on multiple disconnected platforms for academics, placements, examinations, clubs, events, library services, hostel management, grievances, and administrative processes. Students and faculty often spend significant time switching between these systems to find information, complete tasks, or access essential services.

With the rapid advancement of Autonomous AI Agents, campuses have an opportunity to move beyond traditional chatbots toward intelligent systems capable of reasoning, planning, collaborating, and taking actions on behalf of users.

Your challenge is to design and develop a Multi-Agent AI System that acts as a unified smart campus assistant, capable of understanding complex user requests, coordinating specialized AI agents, retrieving institutional knowledge, and autonomously executing tasks across multiple campus services.

The focus of this challenge is not on building another chatbot. Instead, participants are expected to demonstrate agentic AI through autonomous planning, multi-agent collaboration, reasoning, memory, and tool usage.

The Challenge

Build a working proof-of-concept Multi-Agent AI System where multiple specialized AI agents collaborate to solve real-world campus problems.

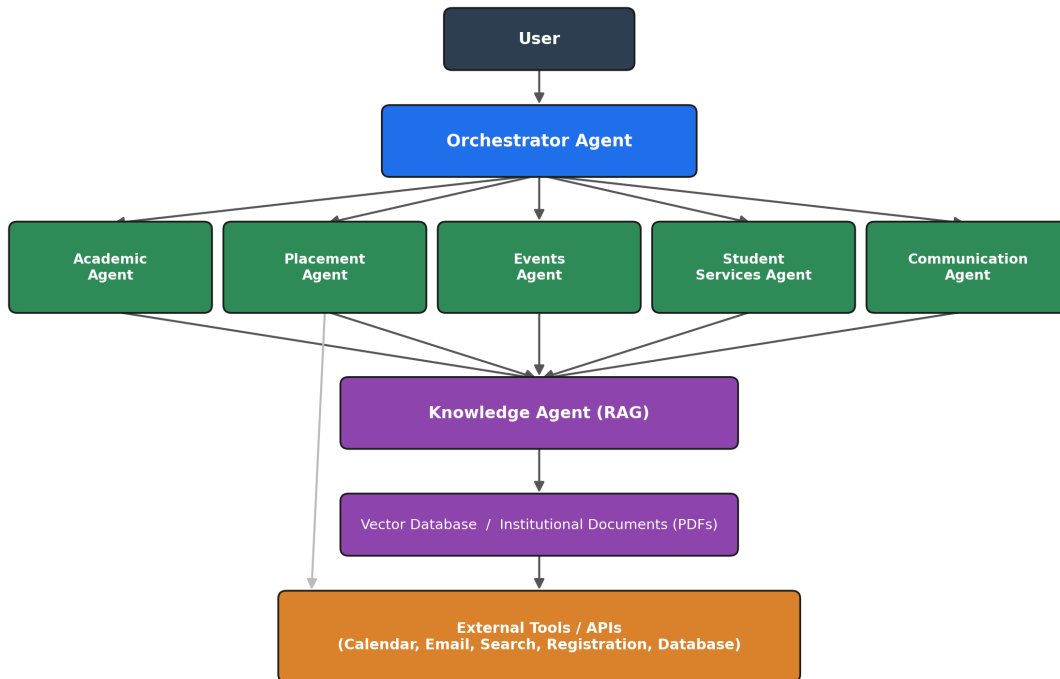
Your solution should include an Orchestrator Agent (or equivalent coordination mechanism) that intelligently routes tasks to one or more specialized agents. These agents should collaborate to complete user requests by retrieving information, reasoning over multiple steps, invoking tools or APIs, and generating meaningful responses.

Participants are encouraged to think beyond simple question answering and demonstrate how AI agents can autonomously plan, coordinate, and execute workflows.



Illustrative Architecture (One Possible Approach)

Smart Campus Multi-Agent AI System — Reference Architecture



Note: This is a reference architecture. Teams are free to design their own multi-agent architecture.

Example Specialized Agents

Participants may build any combination of agents based on their solution.

Academic Agent

- Course information
- Timetables
- Attendance tracking
- Examination schedules
- Academic regulations
- Elective recommendations

Placement Agent

- Internship opportunities
- Company eligibility
- Resume analysis
- Interview preparation
- Placement notifications



HackerRank Campus Crew

Events Agent

- Discover workshops
- Register participants
- Manage hackathons
- Calendar integration
- Event reminders

Student Services Agent

- Hostel information
- Library services
- Scholarships
- Transport
- Grievances
- Campus FAQs

Communication Agent

- Draft emails
- Send notifications
- Generate announcements
- Appointment scheduling

Knowledge Agent

- RAG over institutional documents
- Policies
- Handbooks
- Circulars
- Notices
- FAQs

Notification / Calendar Agent

- Schedule reminders
- Calendar updates
- Event notifications

Illustrative User Workflows

Example 1

"I'm a third-year CSE student. Am I eligible for the Google internship? If yes, register me for tomorrow's placement workshop, add it to my calendar, and remind me one hour before the event."

- Orchestrator understands intent
- Placement Agent checks eligibility
- Knowledge Agent retrieves placement policy
- Events Agent registers the student
- Calendar Agent schedules the event
- Notification Agent creates reminder
- Final response returned to user

Example 2

"Summarize the examination regulations, calculate my attendance eligibility, and draft an email requesting permission for a makeup exam."



HackerRank Campus Crew

Example 3

“Show today’s classes, recommend upcoming AI workshops, and suggest clubs related to Machine Learning.”

Functional Requirements

Your solution should demonstrate more than four of the following:

- Natural language understanding
- Multi-agent collaboration
- Autonomous task planning
- Agent-to-agent communication
- Retrieval-Augmented Generation (RAG)
- Tool / Function Calling
- Workflow orchestration
- Personalized recommendations
- Context-aware conversations
- Memory across interactions
- End-to-end workflow execution
- Error handling and graceful fallback mechanisms

Non-Functional Requirements

Solutions should aim to demonstrate:

- Modular architecture
- Scalable agent design
- Low-latency responses
- Secure handling of user information
- Reliable workflow execution
- Clean and intuitive user experience
- Well-documented architecture



HackerRank Campus Crew

Technical Guidelines

Participants are free to choose any technology stack. Possible technologies include:

AI Frameworks	LangGraph • CrewAI • OpenAI Agents SDK • AutoGen • LangChain • Semantic Kernel • Google ADK • Custom frameworks
LLMs	OpenAI GPT • Gemini • Claude • Llama • Mistral • Qwen • DeepSeek • Other open-source models
Vector Databases	FAISS • ChromaDB • Pinecone • Milvus • Weaviate
Databases	PostgreSQL • MongoDB • SQLite • Firebase • Supabase
Frontend	React • Next.js • Streamlit • Gradio • Flutter • Angular • Any equivalent framework

Dataset & Integrations

Participants are NOT expected to integrate with actual institutional systems. Solutions may use:

- Mock APIs
- JSON datasets
- CSV files
- SQLite / PostgreSQL databases
- Sample institutional documents
- Public datasets
- Simulated campus services

The emphasis is on AI reasoning and autonomous workflows, not enterprise integrations.

Mandatory AI Components

Every solution should include at least three of the following:

- Multi-Agent Architecture
- Retrieval-Augmented Generation (RAG)
- Function / Tool Calling
- Autonomous Planning
- Agent Memory



HackerRank Campus Crew

- Multi-step Reasoning
- Workflow Orchestration
- External API Integration

Stretch Goals (Bonus)

Additional consideration may be given to solutions featuring:

- Voice interaction
- Multilingual support
- Long-term memory
- Human-in-the-loop approval
- Vision capabilities (OCR / Image understanding)
- MCP (Model Context Protocol)
- Agent-to-Agent (A2A) communication
- Real-world API integrations
- Explainable AI
- Mobile application support
- Cloud deployment
- Offline-capable AI models

Expected Deliverables

- Working prototype
- Source code repository
- Architecture diagram
- Brief documentation (README)
- Demo video (if required by organizers)
- Live demonstration during evaluation

Judging Considerations

Judges may evaluate:

- How effectively agents collaborate
- Whether the system demonstrates autonomous decision-making
- Quality of reasoning and planning
- Effective use of AI technologies
- Real-world applicability



Vasavi College Of Engineering (Autonomous)
Accredited by NAAC with A++ Grade



HackerRank Campus Crew

- Robustness and scalability
- User experience
- Overall technical excellence

What We Are Looking For

We are looking for solutions that move beyond traditional chatbots. Successful submissions should demonstrate how multiple AI agents can intelligently collaborate to understand user intent, retrieve relevant knowledge, interact with tools, make informed decisions, and execute meaningful workflows with minimal user intervention.

Creativity is encouraged. Participants are free to define their own agent architecture, workflows, and implementation approach while addressing the core challenge of building an intelligent Smart Campus Multi-Agent AI System.

Expected Outcome

At the end of the 24-hour hackathon, teams should present a functional proof-of-concept that showcases how Autonomous Multi-Agent AI Systems can transform campus experiences by enabling intelligent assistance, collaborative problem-solving, and workflow automation. The emphasis is on demonstrating innovation, practical AI engineering, and real-world impact through agentic systems rather than building a feature-complete campus platform.